

ECO-Crisis Command: The Crash Course

1. The Core Idea: What is this project?

Imagine a modern 911 dispatch center during a massive hurricane. People are calling in fires, floods, and medical emergencies all at the same time. Human dispatchers are overwhelmed. They only have 5 ambulances and 3 fire trucks, but they have 20 emergencies. Who gets help first?

ECO-Crisis Command is a software platform that solves this. It replaces the overwhelmed human dispatcher with a team of Smart AI Agents. The AI looks at the map, sees all the emergencies, counts the available ambulances, and instantly does the math to send the right vehicles to the most critical emergencies to save the most lives.

2. How the Software is Built (The Restaurant Analogy)

- The Frontend (The Dining Room): This is what you see on your screen. It is built using React (a tool for building user interfaces) and styled with Tailwind CSS (makes things look sleek and dark). The interactive map you see is powered by MapLibre.
- The Backend (The Kitchen): You don't see this, but it does all the heavy lifting. It is built using Node.js. When a user clicks a button on the screen, a message is sent to the backend kitchen to "cook up" the data.
- The Database (The Pantry): We use PostgreSQL. This is a massive digital filing cabinet where all the data (where the trucks are, where the fires are) is permanently stored.

3. The Two Screens (Who uses this?)

- The User Dashboard (The Civilian): A very simple screen. If a civilian is trapped in a flood, they use this screen to fill out a quick form saying "I need help, it's a flood!" and hit submit.
- The Admin Dashboard (The Commander): This is the cool, complex map screen. The person running the city looks at this. When the civilian hits "submit", the Admin screen instantly flashes red and drops a wave emoji onto the map.

4. The "AI Agents" (The Secret Sauce)

The hackathon asked you to build an "Agent System." An Agent is just a piece of AI code that has a specific job. In your backend, you have a simulated 8-Agent Pipeline.

The "Mid-Scenario Change" (The Plot Twist):

The hackathon rubric strictly demands that you show what happens if things go wrong. In your project, there is a built-in scenario where an Evacuation Vehicle breaks down mid-rescue. When this happens, your AI Agents realize the truck is broken, they panic for a millisecond, and then they dynamically re-plan. They steal a rescue team from a lower-priority animal farm emergency and send it to save the trapped humans instead. The UI then prints out text explaining exactly why it made that decision.

5. The "Omnichannel Simulator" (The Extra Feature)

What happens if the hurricane destroys the internet cell towers? The civilian can't open a web browser!

To impress the judges, we built a small panel at the bottom of the Admin Map. It has buttons to simulate receiving an offline text message (SMS), a satellite beacon, or a radio packet. When you click it, a hacker-style terminal decodes the raw offline data and plots it on the map. It proves your system survives internet blackouts.

Summary: Your Exact Demo Flow Step-by-Step

- 1. Show the Map: Open the Admin Dashboard. Explain that this is the global view for the city commander. Point out the emojis.
- 2. Submit a Report: Open the User Dashboard next to it. Submit a fake emergency. Tell the judges to watch the Admin map. It will instantly ping and show the new emergency.
- 3. Explain the AI: Show them how the backend AI automatically assigned a rescue vehicle to that new emergency.
- 4. Trigger the Plot Twist: Show them what happens when a vehicle breaks down. Show the text where the AI explains why it moved a different truck to cover the failure.
- 5. Drop the Mic (Offline Simulator): End the presentation by saying, "Oh, and if the internet goes down? Watch this." Click the Satellite/Radio simulator button, watch the terminal decode it, and show the emergency pop up on the map anyway.